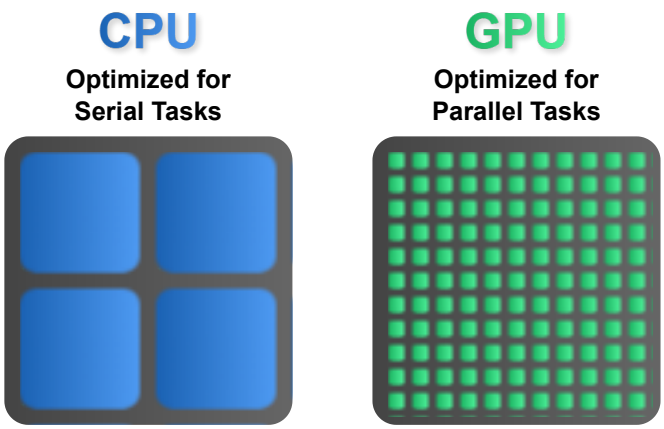
MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Teknologi Kecerdasan Artifisial (Tingkat 1)

Pertemuan I

Kemampuan CPU vs GPU

Pemrosesan data antara penggunaan CPU dan GPU sangat berbeda. Di bawah ini adalah contoh program dengan menggunakan python 3 untuk melihat perbandingan kemampuan antara penggunaan CPU dengan GPU.



Program:

Di bawah ini adalah contoh program python menggunakan library numpy yang memanfaatkan CPU dan juga cupy yang memanfaatkan GPU. Sementara library time digunakan untuk melakukan perhitungan waktu yang diperlukan.

In [1]:
```
import numpy as np
import cupy as cp
import time
```

Kode di bawah ini membuat array 3D dengan jumlah 1 Miliar, untuk Numpy dan CuPy.

In [2]:
```
### Numpy dan CPU
s = time.time()
x_cpu = np.ones((1000,1000,1000))
e = time.time()
print("Waktu yang diperlukan untuk CPU:", e - s)
```

```
Waktu yang diperlukan untuk CPU: 3.2124483585357666
```

In [3]:
```
### CuPy dan GPU
s = time.time()
x_gpu = cp.ones((1000,1000,1000))
cp.cuda.Stream.null.synchronize()
e = time.time()
print("Waktu yang diperlukan untuk GPU:", e - s)
```

```
Waktu yang diperlukan untuk GPU: 0.7608637809753418
```

Dari hasil di atas dapat dilihat perbedaan hasil kecepatan waktu antara penggunaan CPU dan GPU. Program berikutnya untuk melakukan operasi matematika terhadap array yaitu dengan mengkalikannya dengan nilai tertentu.

In [4]:
```
### Nilai yang akan dikalikan dengan array
n = 5
```

In [5]:
```
### Numpy dan CPU
s = time.time()
x_cpu *= n
e = time.time()
print("Waktu yang diperlukan untuk CPU:", e - s)
```

```
Waktu yang diperlukan untuk CPU: 0.8865945339202881
```

In [6]:
```
### CuPy and GPU
s = time.time()
x_gpu *= n
cp.cuda.Stream.null.synchronize()
e = time.time()
print("Waktu yang diperlukan untuk GPU:", e - s)
```

```
Waktu yang diperlukan untuk GPU: 0.17106080055236816
```

#Kesimpulan

Data ini menegaskan bahwa GPU jauh lebih efisien untuk komputasi numerik skala besar, terutama pada tugas-tugas yang dapat diparalelkan seperti operasi element-wise pada array yang masif.

Namun, perlu diingat bahwa tidak semua tugas cocok untuk GPU. Overhead transfer data dan sinkronisasi bisa menjadi penghambat jika data kecil atau komputasi tidak paralel.

Menurut saya pemahaman ini penting untuk memilih arsitektur yang tepat sesuai kebutuhan.

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/ (https://www.praktikum-hpc.gunadarma.ac.id/)

MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Teknologi Kecerdasan Artifisial (Tingkat 1)

Pertemuan I

Program Python untuk Bilangan Fibonacci ke-n

Dalam matematika, bilangan Fibonacci, biasanya dilambangkan dengan `Fn`, membentuk sebuah deret yang disebut deret Fibonacci, sehingga setiap bilangan tersebut merupakan penjumlahan dari dua bilangan sebelumnya, dimulai dari 0 dan 1.

$$F_0 = 0, \quad F_1 = 1,$$

$$F_n = F_{n-1} + F_{n-2}$$

Untuk n > 1. Maka deret yang dihasilkan adalah:

$$0,\ 1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13,\ 21,\ 34,\ 55,\ 89,\ 144,\ \ldots$$

Program:

Tahap pertama untuk membuat program fibonacci yang ke-n, adalah dengan membuat fungsi yang akan dipanggil secara recursive:

In [ ]:
```
def Fibonacci(n):
    if n<0:
        print("Incorrect input")
    # Nilai Fibonacci pertama 0
    elif n==1:
        return 0
    # Nilai Fibonacci kedua 1
    elif n==2:
        return 1
    else:
        return Fibonacci(n-1)+Fibonacci(n-2)
```

Tahap selanjutnya adalah dengan menentukan nilai `n`, yaitu urutan dari deret fibonacci yang ingin diketahui nilainya.

In [ ]:
```
n = 9
```

Langkah terakhir adalah memanggil fungsi yang sudah dibuat sebelumnya dengan memasukkan nilai `n` sebagai parameternya.

In [ ]:
```
print("Bilangan fibonacci yang ke" , n , "adalah" , Fibonacci(n))
```

Tugas

Tampilkan seluruh nilai deret Fibonacci dari mulai 0 sampai ke-n.

In [17]:
```
Fn = int(input("Masukkan nilai n: "))

def Fibonacci(n):
    F0, F1 = 0, 1  # inisialisasi dua suku pertama
    if n >= 1:
        print(F0, end=" ")  # suku pertama + spasi
    if n >= 2:
        print(F1, end=" ")  # suku kedua + spasi

    for i in range(3, n+1):
        F0, F1 = F1, F0 + F1    # loop suku berikutnya yg berupa penambahan dari 2 suku
        print(F1, end=" ")  # sampai suku ke-n

print("Bilangan fibonacci dari F 0 ke F",Fn,"adalah", end=" ")
Fibonacci(Fn)
```

```
Masukkan nilai n: 15
Bilangan fibonacci dari F 0 ke F 15 adalah 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Perhatian! Berikut merupakan cara konversi File IPYNB menjadi File PDF untuk melakukan pengumpulan tugas di Virtual Class. Video konversi (https://youtu.be/qXELXtnAK7c).

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/ (https://www.praktikum-hpc.gunadarma.ac.id/)

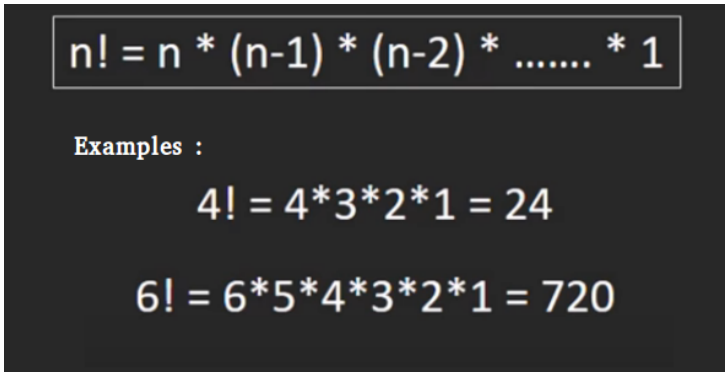
MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Teknologi Kecerdasan Artifisial (Tingkat 1)

Pertemuan I

Program Python untuk Faktorial Bilangan

Faktorial bilangan bulat bukan negatif, adalah perkalian semua bilangan bulat yang lebih kecil dari atau sama dengan n. Misalnya faktorial dari 6 adalah 6 * 5 * 4 * 3 * 2 * 1 yaitu 720.



Program:

Untuk membuat program mencari nilai faktorial dari bilangan bulat bukan negatif, dibagi menjadi 3 tahapan. Tahapan pertama yaitu pendefinisan fungsi faktorial, yang akan dipanggil secara recursive:

In [ ]:
```
def factorial(n):

    return 1 if (n==1 or n==0) else n * factorial(n - 1);
```

Tahap kedua adalah menentukan nilai yang ingin dicari hasil faktorialnya:

In [ ]:
```
num = 5;
```

Tahap ketiga adalah dengan memanggil fungsi yang sudah dibuat sebelumnya dengan menjadikan nilai pada variabel `num` sebagai input parameternya:

In [ ]:
```
print("Factorial of",num,"is", factorial(num))
```

```
Factorial of 5 is 120
```

Tugas

Lakukan pendefinisian fungsi faktorial dengan iterative (menggunakan fungsi perulangan atau iterasi).

In [5]:
```
n = int(input("Masukkan angka: ")) #bisa diubah menjadi input pengguna atau ubah code se

def factorial_iterative(num):
    faktorial = 1
    while num > 0 :
        faktorial *= num
        num -= 1
    return faktorial

print("Factorial of", n, "is", factorial_iterative(n))
```

```
Masukkan angka: 10
Factorial of 10 is 3628800
```

Perhatian! Berikut merupakan cara konversi File IPYNB menjadi File PDF untuk melakukan pengumpulan tugas di Virtual Class. Video konversi (https://youtu.be/qXELXtnAK7c).

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/ (https://www.praktikum-hpc.gunadarma.ac.id/)

MK Praktikum Unggulan Universitas Gunadarma ¶

Mata Kuliah: Praktikum Teknologi Kecerdasan Artifisial (Tingkat 1)

Pertemuan I

Program Python untuk Mencetak Semua Bilangan Prima dalam Sebuah Interval

Definisi: Bilangan prima adalah bilangan asli yang lebih besar dari 1 yang tidak memiliki pembagi positif selain 1 dan dirinya sendiri. Beberapa bilangan prima pertama adalah {2, 3, 5, 7, 11,….}.

Dengan diberikan dua bilangan bulat positif untuk awal dan akhir. Tujuannya adalah menulis program Python untuk mencetak semua bilangan prima dalam Interval tersebut.

Solusi untuk menyelesaikan masalah ini adalah dengan mengulang `val` dari awal ke akhir menggunakan perulangan `for` dan untuk setiap bilangan, jika lebih besar dari 1, periksa apakah bilangan tersebut membagi `n` . Jika ditemukan angka lain yang membagi, cetak nilainya.

Program:

Tahap pertama dalam membuat program ini adalah dengan menentukan nilai awal dan akhir.

In [ ]:
```
awal = 11
akhir = 25
```

Langkah selanjutnya adalah melakukan pengecekan dengan perulangan dan pembagian terhadap nilai interval tersebut.

In [ ]:
```
for i in range(awal, akhir+1):
  if i>1:
    for j in range(2,i):
        if(i % j==0):
            break
    else:
        print(i)
```

```
11
13
17
19
23
```

Tugas

Buatlah sebuah program, untuk menentukan apakah sebuah bilangan merupakan bilangan prima (tanpa

In [4]:
```
n = int(input("Masukkan sebuah bilangan: "))

if n > 1:
    for i in range(2, int(n ** 0.5) + 1): #menentukan bilangan pembagi n dari paling kec
        if n % i == 0: #apabila semua habis, maka bukan bilangan prima
            print(n, "bukan bilangan prima")
            break
    else:
        print(n, "adalah bilangan prima")
else:
    print(n, "bukan bilangan prima")
```

```
Masukkan sebuah bilangan: 11
11 adalah bilangan prima
```

Perhatian! Berikut merupakan cara konversi File IPYNB menjadi File PDF untuk melakukan pengumpulan tugas di Virtual Class. Video konversi (https://youtu.be/qXELXtnAK7c).

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/ (https://www.praktikum-hpc.gunadarma.ac.id/)